

6 EE PROJECTS THAT REACH SPACE FROM YOUR DESK

Satellite downlinks. Attitude control. Precision timing. Cosmic particles. RF, control systems, and low-noise analog. Real EE.

1. GOES Earth Receiver — full-disk planet from 36,000 km up	\$75
2. Meteor-M2 Weather Receiver — image your city from orbit at 137 MHz	\$40
3. Reaction-Wheel Attitude Controller — point a platform like a satellite	\$50
4. Camera Star Tracker — find your orientation from the stars	\$60
5. GPS-Disciplined Oscillator — atomic-grade timing from GPS	\$30
6. Cosmic-Ray Muon Detector — catch particles from space	\$75

PROJECT 1

GOES Earth Receiver

Pull a live, full-disk image of the entire planet off a satellite parked 36,000 km up

Cost: ~\$75

Difficulty: Advanced

Build Time: 2-3 weekends

Why This Matters

GOES (Geostationary Operational Environmental Satellites) are the spacecraft behind every weather forecast you have ever seen. They sit in geostationary orbit 35,786 km above the equator, locked to one spot on Earth, and continuously broadcast imagery back down on an L-band downlink near 1.69 GHz. The operational satellites — GOES-19 (East) and GOES-18 (West) — transmit a signal called HRIT that anyone can receive with a software-defined radio, a low-noise amplifier, and a dish you can build from hardware-store parts. Because the satellite never moves relative to the ground, you aim your antenna once and leave it — no tracking motor. The challenge is pure RF: the signal arrives near the thermal noise floor after a 36,000 km trip, so the whole link budget hinges on a clean, low-noise front end and enough antenna gain to close the link.

What this proves to a recruiter: You understand L-band RF, low-noise amplifier and noise-figure design, link budgets, antenna gain, and digital demodulation of a BPSK downlink. These are the exact skills behind satellite communications, GPS, radar, and every space-to-ground link in the industry.

GOES Earth Receiver — Downlink Signal Chain



Geostationary satellite holds one spot → fixed antenna, no tracking needed
Low-noise front end closes the 36,000 km link → SatDump decodes HRIT to imagery

How It Works

- **Geostationary downlink (1.69 GHz):** GOES-East (GOES-19) and GOES-West (GOES-18) broadcast HRIT near 1694.1 MHz. The signal is BPSK at roughly 400 kSym/s and carries full-disk Earth images, regional mesoscale scans, and weather products.
- **Antenna — gain is everything:** After a 36,000 km path the signal is extremely weak, so you need directional gain. A repurposed 2.4 GHz WiFi grid dish, a small offset satellite dish with a helical feed, or a DIY cookware reflector all work. Because the bird is geostationary, you point once and bolt it down.
- **Low-noise amplifier + SAW filter:** An LNA with a sub-1 dB noise figure, mounted right at the antenna feed, sets the system sensitivity. A SAW filter near 1.69 GHz rejects cellular and WiFi that would otherwise desensitize the receiver. This LNA-at-the-antenna placement is the single most important decision in the build.
- **Bias tee + SDR:** A bias tee injects DC up the coax to power the LNA at the mast while passing RF down to the radio. An RTL-SDR (or Airspy for more headroom) digitizes the filtered signal and streams IQ samples to a laptop or Raspberry Pi.
- **Demodulation + decode (SatDump):** SatDump locks the BPSK carrier, recovers symbol timing, applies Viterbi and Reed-Solomon forward error correction, and reassembles the packets into full-disk and mesoscale images — the same pictures meteorologists use, pulled out of the air by you.

Key Design Decisions

- **Why an LNA at the antenna instead of at the radio?** Coax loss at 1.7 GHz is severe — several dB per meter of cheap cable. Every dB of loss before the first amplifier adds directly to your noise figure, so you amplify before the lossy cable degrades the signal.
- **Why geostationary instead of polar?** A geostationary satellite appears fixed in the sky, so a high-gain, narrow-beam dish can be aimed once and left. A polar satellite races across the sky and would demand a motorized tracker — which is exactly Project 3's problem.
- **Why BPSK for the downlink?** BPSK is the most power-efficient digital modulation — it tolerates the lowest signal-to-noise ratio of any common scheme. When you are link-budget starved from 36,000 km, you trade data rate for the robustness that lets the link close at all.

Bill of Materials

Component	Part / Spec	Qty	~Cost
GOES LNA + SAW filter	1.69 GHz, NF < 1 dB, bias-tee powered	1	\$18
RTL-SDR dongle	RTL2832U + R820T2 (or Airspy Mini)	1	\$30
Bias tee	Inline DC injector, 1–2 GHz	1	\$6
Grid / dish antenna	2.4 GHz WiFi grid or small offset dish	1	\$0–25
Helical / patch feed	DIY copper wire on PCB ground plane	1	\$3
Coax + SMA	Low-loss (LMR-240) pigtails + adapters	1	\$8
Mount + tripod	Fixed-aim hardware	1	\$5
Raspberry Pi / laptop	Runs SatDump	1	—

Build Steps

Build or repurpose the antenna: Start with a 2.4 GHz WiFi grid dish or a small offset satellite dish. Add a feed tuned for 1.69 GHz — a 3–5 turn helical feed wound from copper wire over a ground plane works well and is circularly polarized, matching the satellite. Mount it on a rigid tripod.

Mount the LNA at the feed: Connect the LNA + SAW filter directly to the antenna feed with the shortest possible pigtail — ideally screwed straight onto the connector. This is non-negotiable: amplify before the coax. Weatherproof the joint if it lives outside.

Power it through the bias tee: Insert the bias tee inline at the radio end, feed it 5 V, and verify the LNA draws current. Confirm RF passes to the SDR. A common failure is a bias-tee voltage or polarity mismatch that never powers the LNA — check it with a multimeter.

Aim the dish (once): Look up the azimuth and elevation for GOES-East or GOES-West from your location with any satellite-finder app. Geostationary means a single fixed bearing. Peak the signal in SatDump by nudging the dish a degree at a time until the BPSK shoulders rise out of the noise.

Decode in SatDump: Select the GOES HRIT pipeline, set the sample rate, and start. Once locked you will see the constellation snap to two BPSK points and packets flow. Let a full-disk scan complete and watch a live image of the entire planet assemble on your screen. Save the GeoTIFF and post the timestamp.

Interview Questions:

- Why must the LNA be mounted at the antenna rather than near the receiver?
- What is noise figure, and how does the first amplifier dominate it (Friis formula)?
- How do you compute the link budget for a 36,000 km downlink — what are the dominant losses?
- Why is the GOES downlink BPSK instead of a higher-order modulation like 16-QAM?
- Why does a geostationary satellite allow a fixed antenna while a polar one does not?
- What role do Reed-Solomon and Viterbi coding play in recovering the image error-free?

PROJECT 2

Meteor-M2 Weather Receiver

Image your own city from a satellite screaming overhead at 137 MHz, decoded line by line

Cost: ~\$40

Difficulty: Intermediate

Build Time: 1 weekend

Why This Matters

For years, the way most people received their first satellite image was the NOAA APT signal on 137 MHz — but in 2025 the last NOAA APT satellite was decommissioned and that signal went silent for good. The torch has passed to the Russian Meteor-M series, whose M2-3 and M2-4 satellites transmit a sharper, digital signal called LRPT in the same easy-to-reach 137 MHz band. Unlike geostationary GOES, these are polar-orbiting satellites in low Earth orbit, sweeping over your location a few times a day and painting a strip of the planet beneath them. The whole receive chain costs about the price of a textbook: an RTL-SDR, a simple antenna you can build from two pieces of wire, and free software. It is the most accessible image-from-space project that still actually works in 2026.

What this proves to a recruiter: You understand VHF reception, antenna design and polarization (why satellites use circular polarization), Doppler shift, and digital QPSK demodulation with forward error correction — the same RF fundamentals that scale all the way up to GOES and deep-space communication.

Meteor-M2 Weather Receiver — LRPT Signal Chain



Polar satellite passes overhead → records a strip of Earth beneath its track
SatDump demodulates QPSK LRPT and decodes a high-resolution multi-spectral image

How It Works

- **Polar LRPT downlink (137 MHz):** Meteor-M2-3 and M2-4 transmit LRPT near 137.9 MHz (137.1 MHz backup). Unlike the old analog NOAA APT, LRPT is digital QPSK at about 72 kbps, carrying a crisper multi-spectral image.
- **A satellite that moves:** These satellites orbit roughly 800 km up in a sun-synchronous polar orbit, crossing your sky in about 12–15 minutes per pass. You capture a strip of Earth directly under the ground track — typically your region and a few hundred km either side.
- **Antenna and polarization:** A V-dipole (two ~50 cm wires at about 120 degrees) is the simplest antenna; a QFH (quadrifilar helix) is omnidirectional and better for the ever-changing pass geometry. Satellites transmit circular polarization, which is why a QFH outperforms a flat dipole on low passes.
- **Optional LNA for the city:** In a quiet rural spot the SDR alone can decode a high pass. In a city full of RF noise, a 137 MHz SAW-filtered LNA at the antenna dramatically improves results by rejecting pagers, FM broadcast, and other interference.
- **Doppler and decode (SatDump):** Because the satellite is moving fast, its frequency shifts by several kHz across the pass (Doppler). SatDump tracks the satellite by its orbital elements, corrects Doppler, demodulates the QPSK, applies error correction, and assembles the decoded image.

Bill of Materials

Component	Part / Spec	Qty	~Cost
RTL-SDR dongle	RTL2832U + R820T2 (RTL-SDR Blog V4)	1	\$30
137 MHz SAW LNA	Filtered LNA (1581 marking), bias-tee	1	\$15
Antenna wire	V-dipole legs, ~50 cm each	2	\$3
Coax + connectors	RG-58 + SMA	1	\$5
Bias tee	If LNA is not USB-powered	1	\$4
Tripod / mast	Simple non-metallic support	1	\$3
Laptop / Pi	Runs SatDump + pass prediction	1	—

Build Steps

Build the V-dipole: Cut two wires about 50 cm each (quarter-wave at 137 MHz). Mount them at roughly 120 degrees in a horizontal V pointing up. Connect to coax: one leg to center, one to shield. Drop the coax straight down behind the antenna so it acts as a decoupling stub.

Add the LNA if needed: For urban environments, place the 137 MHz SAW LNA at the antenna and power it via the bias tee. Verify the LNA is the filtered 1581 VHF version — an ADS-B LNA at the wrong frequency will hurt, not help.

Predict a pass: Use a tracker (Look4Sat on Android, or SatDump's built-in tracker) to find the next Meteor-M2-3 or M2-4 pass above about 20 degrees elevation. Note AOS (acquisition of signal) and LOS times. Higher passes give far better images.

Record the pass: In SatDump, select the Meteor M2-x LRPT pipeline (72k mode; try 80k if a newly commissioned bird will not sync). Set gain manually to about 30 dB — do NOT use AGC, which corrupts the narrow LRPT signal. Start a minute before AOS.

Decode and overlay: After the pass, SatDump demodulates and decodes the image. Add map overlays — shorelines and city markers — to prove it is your own region. Compare consecutive passes to watch weather systems move across your area.

Interview Questions:

- Why did NOAA APT reception stop working, and what replaced it?
- Why do weather satellites transmit circular polarization, and how does that affect antenna choice?
- What causes the Doppler shift across a pass, and how is it corrected?
- What is the advantage of digital LRPT (QPSK) over the old analog APT?
- Why must you disable AGC on the SDR for narrowband satellite signals?
- How does a quadrifilar helix achieve an omnidirectional, circularly-polarized pattern?

PROJECT 3

Reaction-Wheel Attitude Controller

A spinning flywheel rotates and holds a platform's heading — how satellites point without thrusters

Cost: ~\$50

Difficulty: Advanced

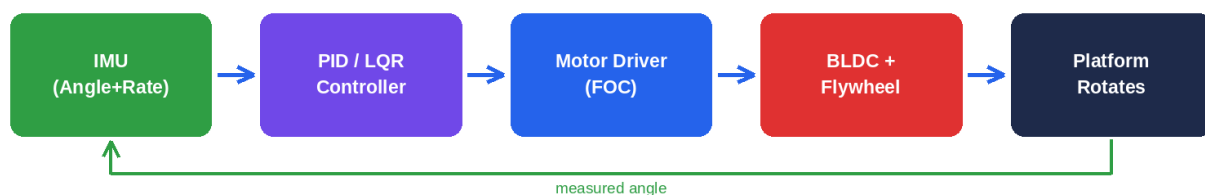
Build Time: 2-3 weekends

Why This Matters

A satellite in orbit has no ground to push against, so how does it turn to point a camera or antenna? Most spacecraft use reaction wheels: spin a flywheel one way and conservation of angular momentum rotates the spacecraft the other way. Speed the wheel up or slow it down and you apply precise torque to the body — no fuel, no thrusters. This project builds a single-axis reaction-wheel platform (or the dramatic reaction-wheel pendulum that balances on an edge): an IMU senses the platform's angle, a control loop computes the torque needed, and a motor accelerates a flywheel to deliver it. It is the cleanest possible demonstration of real attitude control — the same principle flying on every imaging satellite, the Hubble Space Telescope, and the control gyros of the ISS.

What this proves to a recruiter: You understand closed-loop control (PID/LQR), sensor fusion, BLDC motor torque control, and real-time embedded firmware. Attitude determination and control is a core aerospace discipline, and the same fundamentals underpin robotics, drones, and self-balancing vehicles.

Reaction-Wheel Attitude Controller — Control Loop



IMU measures attitude error → controller computes torque → wheel accelerates
Reaction torque rotates the platform — angular momentum, no thrusters

How It Works

- **Conservation of angular momentum:** The platform sits on a low-friction bearing. When the motor accelerates the flywheel clockwise, the platform reacts by rotating counter-clockwise. The torque on the body equals the rate of change of the wheel's angular momentum — so wheel acceleration is your control input.
- **IMU attitude sensing:** An MPU-6050 or BNO055 IMU fuses its gyroscope and accelerometer to estimate the platform's tilt angle and angular rate. A complementary or Kalman filter blends the drift-free-but-noisy accelerometer with the smooth-but-drifting gyro.
- **The control loop:** A PID (or LQR) controller takes the angle error and computes a commanded wheel torque. For the balancing-pendulum version it must stabilize an inherently unstable equilibrium — like balancing a broom on your palm, but a thousand times a second.
- **BLDC torque control:** A brushless gimbal motor spins the flywheel. A field-oriented control (FOC) driver regulates motor current, and since torque is proportional to current, this gives direct, smooth torque control — far better than crude PWM speed control.
- **Momentum saturation:** If the platform keeps disturbing in one direction, the wheel spins faster and faster until it hits its limit — saturation. Real satellites dump this momentum with magnetorquers or thrusters. On the bench you will watch the wheel max out and learn why momentum management matters.

Bill of Materials

Component	Part / Spec	Qty	~Cost
BLDC gimbal motor	2208/2212 brushless, low-kv	1	\$12
FOC driver board	SimpleFOC Mini / DRV8313	1	\$9
IMU	MPU-6050 or BNO055	1	\$4
Microcontroller	ESP32 or STM32 (fast control loop)	1	\$6
Flywheel	Machined / 3D-printed disk + brass rim	1	\$8
Low-friction bearing	Ball bearing for platform pivot	1	\$4
LiPo battery	3S, 11.1 V	1	\$8
Frame	3D-printed / aluminium bracket	1	\$5

Build Steps

Build the mechanics: Mount the BLDC motor on a rigid arm or platform that pivots on a single low-friction bearing. Attach a balanced flywheel to the shaft — mass at the rim maximizes moment of inertia per gram. Balance it so it does not introduce its own torque.

Wire motor + driver: Connect the three BLDC phases to the FOC driver and the driver to the MCU. Power from the LiPo with appropriate current limits. Run the FOC library's alignment routine so the controller learns the rotor's electrical zero.

Bring up the IMU: Mount the IMU rigidly on the platform, not on the vibrating motor. Read gyro and accelerometer over I2C at about 1 kHz. Implement a complementary filter for a clean tilt-angle estimate. Verify the angle tracks reality as you tilt the platform by hand.

Close the loop: Implement PID: $\text{error} = \text{target angle} - \text{measured angle}$; $\text{output} = \text{commanded motor current (torque)}$. Start with proportional gain only, add derivative for damping, then a little integral. Tune until the platform holds its target heading and rejects a finger-push disturbance.

Demonstrate and characterize: Command a step change in heading and film the platform snapping to the new angle. For the pendulum build, show it balancing upright on its edge. Log angle vs. time to show your step response — overshoot, settling time, steady-state error — the language of control engineering.

Interview Questions:

- How does conservation of angular momentum let a reaction wheel rotate a spacecraft?
- Why is motor current (torque) control better than PWM speed control here?
- What is reaction-wheel saturation, and how do real satellites desaturate?
- How does a complementary or Kalman filter fuse gyro and accelerometer data?
- Why is the balancing pendulum unstable, and what does that demand of the controller?
- How would you extend single-axis control to full 3-axis attitude control?

PROJECT 4

Camera Star Tracker

Point it at the night sky — it identifies the stars and outputs your exact 3-axis orientation

Cost: ~\$60

Difficulty: Advanced

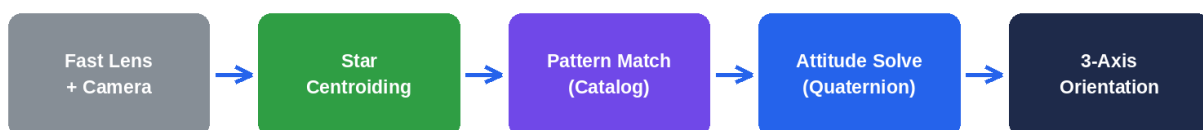
Build Time: 2-3 weekends

Why This Matters

Before a satellite can point at anything, it has to know which way it is already facing — and the most accurate way to do that in space is to look at the stars. A star tracker photographs a patch of sky, identifies the star pattern by matching it against an onboard catalog, and from that computes the spacecraft's precise 3-axis orientation, accurate to arc-seconds. This is the highest-precision attitude sensor ever flown, and you can build a working one: a camera with a fast lens, a small computer, and an open-source lost-in-space plate-solving algorithm that figures out where you are pointed with zero prior information. Pair it with Project 3's reaction wheel and you have both halves of a real spacecraft's attitude system — sensing and actuation.

What this proves to a recruiter: You understand low-light image sensors and optics (f-number, exposure), embedded computer vision, star-identification algorithms, and the coordinate-frame math (quaternions) behind pointing systems. Star trackers are flagship aerospace GNC hardware — building one signals you can work on real spacecraft guidance.

Camera Star Tracker — Attitude Determination



Camera captures stars → centroid each one → match the pattern to a star catalog
Solve for orientation as a quaternion — arc-second pointing from the sky alone

How It Works

- **Fast lens + low-light sensor:** Stars are dim, so the optics matter most. A fast lens ($f/1.4$ – $f/2$) gathers maximum light; a sensitive CMOS sensor (a Raspberry Pi HQ camera or a Sony IMX module) with a multi-second exposure captures enough stars per frame to identify a pattern. Monochrome and longer exposure both help.
- **Star centroiding:** The software thresholds the image to separate stars from background, then computes the sub-pixel centroid of each star blob. Centroiding to a fraction of a pixel is what gives the tracker its arc-second accuracy — far finer than the pixel grid itself.
- **Pattern matching (lost in space):** The key trick: the angular distances between stars form a unique fingerprint. An open-source solver like Tetra3 hashes these geometric patterns and matches them against a precomputed catalog — identifying exactly which stars you are looking at with no prior orientation estimate.
- **Attitude solve:** Once stars are identified, their known catalog positions (right ascension and declination) plus their measured pixel positions let the solver compute the rotation that maps one to the other. The result is the camera's orientation, expressed as a quaternion or rotation matrix.
- **Edge compute:** All of this runs on a Raspberry Pi (Zero 2 W or Pi 4). The solve takes well under a second per frame — fast enough to output a continuous attitude estimate, exactly as a real star tracker feeds a spacecraft's control system.

Bill of Materials

Component	Part / Spec	Qty	~Cost
Raspberry Pi	Zero 2 W or Pi 4 (runs Tetra3)	1	\$15
Camera module	Pi HQ camera or IMX290 / 462 board	1	\$22
Fast lens	CCTV C-mount, $f/1.4$ – $f/2$, ~16 mm	1	\$12
Lens adapter	C / CS-mount to camera	1	\$3
MicroSD card	OS + star catalog	1	\$5
Mount	3D-printed rigid camera bracket	1	\$3
Power	USB-C / LiPo + buck	1	—

Build Steps

Assemble the optics: Fit the fast C-mount lens to the camera via the adapter. Focus at infinity on a distant light at night — the sharpest star points give the best centroids. A rigid, vibration-free mount is essential; any blur smears stars and kills accuracy.

Set up the compute: Flash the Pi and install Python, OpenCV, and the Tetra3 star-identification library. Download or generate the star-catalog database tuned to your lens's field of view (Tetra3 builds this from a catalog such as Hipparcos).

Capture night-sky frames: Take test exposures of a few seconds pointed at a clear sky. Tune exposure and gain until you reliably capture a dozen-plus stars per frame without saturating the brightest ones. Confirm stars appear as crisp points, not streaks.

Run the solver: Feed a frame to Tetra3. It centroids the stars, matches the pattern, and returns the celestial coordinates the camera is pointed at, plus a roll angle. Verify against a planetarium app (Stellarium) aimed at the same patch of sky.

Output continuous attitude: Loop the capture-and-solve at a steady cadence, printing the live quaternion. Mount the tracker on Project 3's reaction-wheel platform and you have closed the full loop: the star tracker says where you point, the wheel turns you where you want.

Interview Questions:

- Why does f-number matter more than megapixels for a star tracker?
- How does sub-pixel centroiding achieve accuracy finer than the pixel pitch?
- What is the lost-in-space problem and how does pattern matching solve it without a prior?
- Why are inter-star angular distances used as the matching feature?
- How is orientation represented, and why are quaternions preferred over Euler angles?
- How would a star tracker and a reaction wheel work together in a real spacecraft?

PROJECT 5

GPS-Disciplined Oscillator

Lock a local oscillator to the atomic clocks aboard GPS satellites for parts-per-billion accuracy

Cost: ~\$30

Difficulty:
Intermediate-Advanced

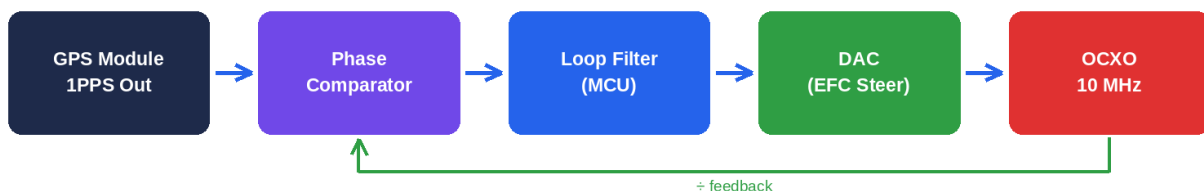
Build Time: 1-2 weekends

Why This Matters

Every GPS satellite carries an atomic clock, and the timing signal it broadcasts is disciplined to within nanoseconds of the global time standard. A GPS-disciplined oscillator borrows that accuracy: it takes the once-per-second pulse (1PPS) from a cheap GPS receiver and uses it to steer a local crystal oscillator so its frequency is locked, long-term, to atomic precision — parts-per-billion or better. This is the quiet workhorse of every serious RF lab, cellular base station, and test bench: the reference that everything else is measured against. Building one teaches you precision timing, control loops, and the discipline of measuring frequency accurately — and it is the most affordable project in this guide.

What this proves to a recruiter: You understand precision frequency references, phase-locked control loops, GNSS timing (1PPS), oscillator steering via DAC or varactor, and how to characterize stability with Allan deviation. Timing and synchronization is foundational to communications, radar, and instrumentation — and surprisingly few engineers actually understand it.

GPS-Disciplined Oscillator — Timing Loop



GPS 1PPS = atomic-clock reference → compared against the divided oscillator
MCU steers the oscillator via DAC until it locks to GPS — parts-per-billion accuracy

How It Works

- **The 1PPS reference:** A GPS receiver outputs a one-pulse-per-second signal whose rising edge is aligned to GPS system time — itself steered by the atomic clocks aboard the satellites. The edge is accurate to tens of nanoseconds, but it jitters pulse-to-pulse, so it is a great long-term reference and a poor short-term one.
- **The local oscillator:** An OCXO (oven-controlled crystal oscillator) is extremely stable second-to-second but slowly drifts over hours and temperature. It is the opposite of GPS: great short-term, poor long-term. The whole idea is to combine the two.
- **Phase comparison:** The MCU divides the OCXO (e.g. 10 MHz) down to 1 Hz and measures the time error between that and the GPS 1PPS using a hardware timer's input capture. This error, in nanoseconds, is the signal the control loop drives to zero.
- **The loop filter:** A slow software control loop — a long time constant of many seconds to minutes — averages out the GPS jitter and computes a correction. Too fast and you chase GPS noise; too slow and you fail to track drift. This tradeoff is the heart of the design.
- **Steering the oscillator:** The correction drives a DAC that sets the OCXO's electronic frequency control (EFC) voltage, nudging it a few parts-per-billion. Over minutes the loop converges and the OCXO is disciplined — its frequency now traceable to the same standard as the satellites overhead.

Bill of Materials

Component	Part / Spec	Qty	~Cost
GPS module	u-blox NEO-M8N / 6M with 1PPS out	1	\$10
OCXO	10 MHz, EFC input (surplus ok)	1	\$12
DAC	MCP4725 / 16-bit I2C DAC	1	\$3
Microcontroller	STM32 (timer input capture)	1	\$4
GPS antenna	Active patch antenna	1	\$4
Voltage reference	Low-noise rail for the DAC	1	\$2
OLED display	Show lock status + error	1	\$4

Build Steps

Get a GPS fix + 1PPS: Wire the GPS module and antenna; confirm it acquires satellites and outputs a clean 1PPS. Scope the pulse — it should be rock-steady at 1 Hz once locked. Place the antenna with a clear sky view.

Run the OCXO free: Power the OCXO and let its oven stabilize for a few minutes. Feed its 10 MHz into an MCU timer and measure its frequency against the 1PPS, uncorrected, first. You will see it is off by a small, slowly changing amount — that drift is what you will cancel.

Measure phase error: Use the MCU's input-capture timer to timestamp both the divided OCXO edge and the GPS 1PPS edge. The difference is your phase error in nanoseconds. Stream it to the OLED and watch it wander as the OCXO drifts.

Close the discipline loop: Implement a slow PI control loop: integrate the averaged phase error and output a DAC value to the OCXO's EFC pin. Choose a long time constant (tens of seconds) so GPS jitter is averaged out. Watch the error converge toward zero and stay there.

Characterize stability: Log the phase error over hours and compute the Allan deviation — the standard measure of oscillator stability. Show the disciplined OCXO beats both the free-running OCXO (long-term) and raw GPS (short-term). That crossover plot is the money shot.

Interview Questions:

- Why combine a GPS receiver and an OCXO — what does each contribute?
- Why is the 1PPS a good long-term reference but a poor short-term one?
- How do you measure the phase error between the oscillator and the 1PPS?
- How does the loop time constant trade jitter rejection against drift tracking?
- What is Allan deviation and why is it the right metric for oscillator stability?
- What is holdover, and how should the loop behave when GPS signal is lost?

PROJECT 6

Cosmic-Ray Muon Detector

Every flash is a particle born from a cosmic ray hitting the atmosphere, passing through your desk

Cost: ~\$75

Difficulty: Advanced

Build Time: 2-3 weekends

Why This Matters

Right now, particles created when cosmic rays from exploding stars and distant galaxies slam into the upper atmosphere are raining down through your body — about one muon through every square centimeter every minute. You can build an instrument that catches them. When a muon passes through a plastic scintillator it leaves a tiny flash of light; a silicon photomultiplier (SiPM) turns that flash into an electrical pulse, a fast amplifier boosts it, and a discriminator counts it. This is real particle-physics instrumentation — the same detection principle used at CERN — built on your bench for under \$80. It is also the deepest analog challenge in this guide: catching a microsecond flash of a few hundred photons demands genuinely low-noise, high-speed front-end design.

What this proves to a recruiter: You understand SiPM biasing and temperature compensation, low-noise transimpedance amplification, fast pulse discrimination, and coincidence detection. This is high-speed, low-noise analog at the edge of what is possible — exactly the skill set behind medical imaging, LIDAR, and scientific instrumentation.

Cosmic-Ray Muon Detector — Detection Chain



Muon crosses scintillator → nanosecond flash of photons → SiPM makes a current pulse
Amplify, threshold above noise, and count — every tick is a particle from space

How It Works

- **The scintillator:** A block of plastic scintillator (cast PVT, or a surplus offcut) emits a brief flash of blue light when a charged particle deposits energy crossing it. A single muon produces only a few hundred to a few thousand photons — faint and fast, lasting nanoseconds.
- **Silicon photomultiplier (SiPM):** A SiPM is an array of avalanche photodiodes biased just above breakdown, giving enormous gain (around 10^6) — enough to turn a handful of photons into a measurable current pulse. It replaces the bulky, high-voltage photomultiplier tubes of old with a 6 mm chip running on about 29 V.
- **Precise bias + temperature:** A SiPM's gain is steeply sensitive to bias voltage and temperature, so a stable, low-noise bias supply (a quiet boost converter plus filtering) is critical. Drift the voltage and your count rate drifts with it — which is why real systems add temperature compensation.
- **Fast amplifier:** A transimpedance or voltage amplifier with enough bandwidth (tens of MHz) converts the SiPM's fast current spike into a clean voltage pulse without smearing it. Layout matters here — short traces, good grounding, and supply bypassing keep noise below the tiny signal.
- **Discrimination and counting:** A comparator fires when a pulse exceeds a threshold set just above the noise floor, producing a clean digital tick per event. An MCU counts ticks and shows the muon rate on an OLED. Add a second detector and require coincidence — only count when both fire at once — to reject electrical noise and confirm real through-going particles.

Bill of Materials

Component	Part / Spec	Qty	~Cost
SiPM	MicroFC-60035 or equivalent 6 mm	1	\$30
Plastic scintillator	Cast PVT block or offcut	1	\$15
Fast op-amp	Wide-band (OPA / AD8000-class)	1	\$4
Comparator	Fast (LT1716 / LM361-class)	1	\$2
Boost converter	Adjustable ~29 V SiPM bias	1	\$4
Arduino Nano	Count + OLED drive	1	\$5
OLED display	128x64 I2C	1	\$4
Light-tight enclosure	Black box / foil-lined	1	\$3

Build Steps

Couple SiPM to scintillator: Wrap the scintillator in reflective foil (Tyvek or aluminized mylar), leaving one face for the SiPM. Optically couple the SiPM to that face with optical grease or a clear pad. Seal everything in a light-tight enclosure — ambient light will swamp the SiPM instantly.

Build the bias supply: Use an adjustable boost converter for the SiPM bias (about 29 V; check your SiPM's breakdown plus overvoltage spec). Filter it heavily with RC stages — any ripple shows up as noise. Set the exact voltage with a multimeter before connecting the SiPM.

Amplify the pulse: Build the fast amplifier on a compact board with a ground plane. Keep the SiPM-to-amp path short. Scope the output: cosmic muons appear as occasional fast spikes a few times a second above the noise.

Set the discriminator: Feed the amplified signal to the comparator. Set the threshold just above the noise floor — high enough to ignore noise, low enough to catch real pulses. Each crossing generates one clean digital pulse to the MCU.

Count and (optionally) coincidence: Program the MCU to count pulses and show the muon rate on the OLED. Expect very roughly one count per cm^2 per minute of detector area. For a rigorous version, build a second detector and only count simultaneous hits — coincidence proves you are seeing real particles, not noise.

Interview Questions:

- How does a scintillator plus SiPM convert a passing particle into an electrical pulse?
- Why is a SiPM's bias voltage so critical, and how does temperature affect its gain?
- What bandwidth does the amplifier need, and why does layout matter for low-noise design?
- How do you set the discriminator threshold relative to the noise floor?
- Why does coincidence detection between two scintillators reject false counts?
- Roughly what muon flux do you expect at sea level, and how would you measure efficiency?

Every one of these started as aerospace and defense hardware.

Now it fits on your bench.

Satellite downlinks. Attitude control. Precision timing. Particle detection. These are not toy demos — they are the actual subsystems of spacecraft and the instruments of space science, rebuilt with a software-defined radio, a microcontroller, and parts that cost less than a textbook. The hard parts are real: closing a link budget from 36,000 km, stabilizing an unstable control loop a thousand times a second, catching a nanosecond flash of a few hundred photons. That difficulty is the point.

Aerospace, defense, and the new commercial-space boom are hiring engineers who can actually build this hardware — not just describe it. A working demo of any one of these puts you in a different conversation.

Build even one of these. Log the link budget, screenshot the decoded image, plot the step response or the Allan deviation. Put it on your GitHub with a proper README. You will have a portfolio piece that makes an aerospace interviewer lean forward.

Follow [@1nska](#) on Instagram for build logs, new guides, and projects that reach the edge of space.

— Enes (@1nska) • Electrical Engineer